

EX.NO.06
28.02.25

SET OPERATION,AGGREGATE FUNCTION & GROUP BY

AIM:

To perform aggregation on the relational database and group by records based on the conditions and to perform set operations.

TABLES:

```
SQL> select * from salary;
```

EMP_	ENAME	ACC_	SALARY
E001	Arun	ac01	50000
E002	viji	ac02	45000
E003	yokarajan	ac03	60000
E004	vishal	ac04	10000
E005	sudee	ac05	15000

```
SQL> select * from tax;
```

TAX_	NAME	ACC_	TAX_AMOUNT
T001	Arun	ac01	5000
T002	ram	ac08	4500
T003	yokarajan	ac03	6000
T004	Rose	ac07	7000

SET OPERATION:

UNION:

```
SQL> select ename from salary union select name from tax;
```

ENAME

Arun

Rose

ram

sudee

viji

vishal

yokarajan

7 rows selected.

UNION ALL:

SQL> select ename from salary union all select name from tax;

ENAME

Arun

viji

yokarajan

vishal

sudee

Arun

ram

yokarajan

Rose

9 rows selected.

INTERSECTION:

SQL> select ename, acc_no from salary intersect select name, acc_no from tax;

ENAME	ACC_
-------	------

Arun	ac01
------	------

yokarajan	ac03
-----------	------

EXCEPT or MINUS:

```
SQL> select ename from salary minus select name from tax;
```

ENAME

sudee

viji

vishal

```
SQL> select name from tax minus select ename from salary;
```

NAME

Rose

Ram

AGGREGATE FUNCTION:

```
SQL> select * from staff;
```

ID	NAME	DEPT	SALARY	AGE
101	Arun	CSE	50000	35
102	Bala	ECE	60000	40
103	Yokarajan	IT	55000	27
104	Ram	IT	58000	42
105	Raju	EEE	40000	45
106	Dharun	CHEM	35000	24

6 rows selected.

AVG:

```
SQL> select avg(salary) as average_salary from staff;
```

AVERAGE_SALARY

49666.6667

MAX:

```
SQL> select max(salary) as max_salary from staff;
```

```
MAX_SALARY
```

```
-----
```

```
60000
```

MIN:

```
SQL> select min(salary) as min_salary from staff;
```

```
MIN_SALARY
```

```
-----
```

```
35000
```

SUM:

```
SQL> select sum(salary) as total_salary from staff;
```

```
TOTAL_SALARY
```

```
-----
```

```
298000
```

COUNT:

```
SQL> select count(name) as total_emp from staff;
```

```
TOTAL_EMP
```

```
-----
```

```
6
```

AGGEREGATE FUNCTION WITH GROUPBY,AND HAVING:

```
SQL> select * from staff;
```

```
      ID NAME      DEPT      SALARY      AGE
```

```
-----
```

```
    101 Arun      CSE      50000      35
```

102	Bala	ECE	60000	40
103	Yokarajan	IT	55000	27
104	Ram	IT	58000	42
105	Raju	EEE	40000	45
106	Dharun	CHEM	35000	24

6 rows selected.

GROUP BY:

SQL> select dept, avg(salary) as avg_sal from staff group by dept;

DEPT	AVG_SAL
IT	56500
EEE	40000
CSE	50000
CHEM	35000
ECE	60000

GROUP BY WITH HAVING CLAUSE:

SQL> select dept, avg(salary) as avg_sal from staff group by dept having avg(salary) >= 55000;

DEPT	AVG_SAL
IT	56500
ECE	60000

DISTINCT:

SQL> select distinct dept from staff;

DEPT
IT
EEE

CSE

CHEM

ECE

SECOND HIGHEST SALARY:

```
SQL> select max(salary) as second_high_sal from staff where salary not in
(select max(salary) from staff);
```

SECOND_HIGH_SAL

58000

SECOND LOWEST SALARY:

```
SQL> select min(salary) as second_lowest_sal from staff where salary not
in (select min(salary) from staff);
```

SECOND_LOWEST_SAL

40000

**FINDING THE NAME OF THE PERSONS WHO HAVE HIGHER SALARY THAN THE AVERAGE
SALARY OF THEIR DEPARTMENT:**

```
SQL> select name from staff where salary >= all (select avg(salary) from
staff group by dept);
```

NAME

Bala

CONTENTS	MARKS ALLOTED	MARKS OBTAINED
Aim, Algorithm, SQL, PL/SQL	30	
Execution and Result	20	
Viva	10	
Total	60	

RESULT:

Thus various set operation, aggregate function, group by and having clause operations are performed.